roles based on our needs: for example, in a cluster of 20 nodes, we can enable 3 nodes as master nodes, 15 as data nodes, and 2 as ingest nodes.

To configure a role on a node, all we need to do is tweak the `node.roles` setting in the elasticsearch.yml configuration file. The setting takes in a list of roles: for example, setting `node.roles: [master]` enables the node as a master node. Multiple node roles can be set, as shown in the following example:

```
node.roles: [master, data, ingest, ml]
```

This node has four roles: master, data, ingest, and machine learning.

Remember, we mentioned that the coordinator role is the default role provided to all nodes. Although we set up four roles in the example (`master`, `data`, `ingest`, and `ml`), this node still inherits a `coordinator` role.

We can specifically assign just a `coordinator` role to a node by simply omitting any `node.roles` values. In the following snippet, we assign the node nothing but the `coordinator` role, meaning this node doesn't participate in any activities other than coordinating requests:

```
nodes.roles : [ ]
```

Leaving the roles array empty sets the node as a coordinator.

There is a benefit to enabling nodes as dedicated coordinators: they perform as load balancers, working through requests and collating the result sets. However, the risk outweighs the benefits if we enable many nodes as just coordinators.

In our earlier discussions, I mentioned that Elasticsearch stores analyzed full-text fields in an advanced data structure called an inverted index. If there's one data structure that any search engine (not just Elasticsearch) depends on heavily, that's the inverted index. It is time to examine the internal workings of an inverted index, to solidify our understanding of the text analysis process, storage, and retrieval.

3.3 Inverted indexes

If you look at the back of any book, usually you'll find an index that maps keywords to the pages where they are found. This is nothing but a physical representation of an inverted index.

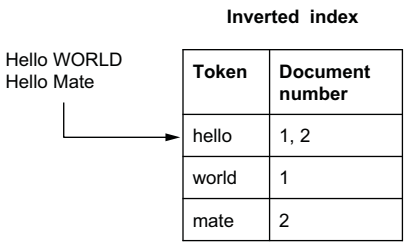


Figure 3.17 An inverted index data structure

Similar to navigating to pages based on the page numbers associated with the keyword you are searching for in a book, Elasticsearch consults the inverted index for search words and document associations. When the engine finds document identifiers for these search words, it returns the full document(s) by querying the server to return them to the client. Elasticsearch uses a data structure called an *inverted index* for each full-text field during the indexing phase, as figure 3.17 illustrates.